

UNIVERSIDADE FEDERAL DE PERNAMBUCO
CENTRO DE INFORMÁTICA
CIÊNCIAS DA COMPUTAÇÃO

ARTHUR HENRIQUE ANIBAL DA COSTA

ANÁLISE COMPARATIVA ENTRE INGESTÃO DE DADOS EM BATCH E STREAM EM AMBIENTE SIMULADO PARA INDÚSTRIA 4.0

Recife
2026

ARTHUR HENRIQUE ANIBAL DA COSTA

ANÁLISE COMPARATIVA ENTRE INGESTÃO DE DADOS EM BATCH E STREAM EM AMBIENTE SIMULADO PARA INDÚSTRIA 4.0

Trabalho de Conclusão de Curso apresentado ao Curso de Ciência da Computação da Universidade Federal de Pernambuco, como requisito parcial para obtenção do título de Bacharelado em Ciência da Computação.

Orientador (a): Kiev Santos da Gama

Recife

2026

ARTHUR HENRIQUE ANIBAL DA COSTA

ANÁLISE COMPARATIVA ENTRE INGESTÃO DE DADOS EM BATCH E STREAM EM AMBIENTE SIMULADO PARA INDÚSTRIA 4.0

Trabalho de Conclusão de Curso apresentado ao Curso de Ciência da Computação da Universidade Federal de Pernambuco, como requisito parcial para obtenção do título de Bacharelado em Ciência da Computação.

Aprovada em: XX/XX/20XX

BANCA EXAMINADORA

_____

Prof. Dr. Kiev Santos da Gama (Orientador)
Universidade Federal de Pernambuco

_____

Prof. Dr. Texto Texto Texto (Examinador Interno)
Universidade Federal de Pernambuco

Dedico esse trabalho a todos aqueles que quase desistiram e ainda assim fizeram coisas maravilhosas. Eu vejo vocês.

AGRADECIMENTOS

Gostaria de agradecer à todos aqueles que sempre demonstraram se preocupar comigo. Aos incentivos que me deram e palavras de apoio. Em especial agradeço aos meus pais, que serviram de alicerce para ser o que sou hoje e em todos os momentos me fizeram sentir confiante para continuar os estudos.

RESUMO

Este trabalho propõe uma análise comparativa entre duas abordagens de ingestão de dados amplamente utilizadas em sistemas analíticos modernos: *batch* e *streaming*. A partir de uma simulação de ambiente industrial com 33 máquinas e 78 sensores, foram realizados experimentos controlados para avaliar o comportamento de ambas as estratégias sob diferentes níveis de carga.

Foram analisadas métricas críticas como latência, throughput, uso de CPU, uso de memória e operações de entrada e saída. Os resultados revelam um contraste marcante entre os modelos: enquanto o *batch* apresenta maior latência e consumo esporádico de recursos, o *streaming* exige alocação contínua, mas entrega dados com baixa latência, mesmo sob variação de volume.

Mais do que uma simples comparação, este estudo evidencia como essas abordagens podem ser complementares, cada uma mais adequada a contextos específicos. Ao longo deste trabalho, o leitor encontrará dados, gráficos e interpretações que elucidam o funcionamento prático dessas estratégias e que podem auxiliar na escolha da solução ideal conforme o perfil da aplicação.

Palavras-chave: Ingestão; Processamento; Real-time; Batch; Métricas; Comparativo.

ABSTRACT

This study presents a comparative analysis between two widely adopted data ingestion strategies in analytical systems: batch and streaming. Based on an industrial simulation environment with 33 machines and 78 sensors, controlled experiments were conducted to assess the performance of both strategies under varying data loads.

Key metrics such as latency, throughput, CPU usage, memory consumption, and I/O operations were analyzed. The results reveal a sharp contrast between the two models: while batch processing exhibits higher latency and sporadic resource usage, streaming requires continuous allocation but ensures low-latency delivery even under increased workload.

Rather than pointing to a definitive winner, this study highlights how each strategy may be better suited to specific use cases. Throughout the paper, the reader will find data, graphs, and insights that shed light on the practical behavior of these approaches supporting more informed decisions when choosing between real-time and batch-oriented data processing models.

Keywords: Ingestion; Processing; Real-time; Batch; Metrics; Comparative.

SUMÁRIO

AGRADECIMENTOS **6**
RESUMO **7**
ABSTRACT **8**
1. INTRODUÇÃO **14**
2. FUNDAMENTAÇÃO TEÓRICA **16**
2.1. INTERNET DAS COISAS E O DESAFIO DA INGESTÃO DE DADOS 16
2.2. INGESTÃO DE DADOS: CONCEITO E CLASSIFICAÇÕES 16
2.3. PROCESSAMENTO 18
2.4. ARQUITETURAS HÍBRIDAS: KAPPA E LAMBDA 18
3. TRABALHOS RELACIONADOS **20**
3.1. INTEGRAÇÃO ENTRE PROCESSAMENTO BATCH E STREAM 20
3.2. ESTUDOS EM AMBIENTES IOT 20
3.3. LIMITAÇÕES OBSERVADAS NOS TRABALHOS EXISTENTES 21
4. METODOLOGIA **21**
4.1. MÉTRICAS DE AVALIAÇÃO 22
4.2. FERRAMENTAS E PARÂMETROS UTILIZADOS 24
4.3. DESIGN DO EXPERIMENTO 25
4.3.1. Simulação da planta industrial 26
4.3.2. Ingestão dos dados 26
4.3.3. Coleta de métricas e instrumentação 27
4.3.4. Execução com diferentes cargas de trabalho 27
5. DISCUSSÃO DOS RESULTADOS **27**
5.1. COMPARATIVO ENTRE ABORDAGENS COM CARGA DE 1x 27
5.2. COMPARATIVO ENTRE ABORDAGENS - CARGA DE 2X 32
5.3. COMPARATIVO ENTRE ABORDAGENS - CARGA DE 4X 33
5.4. ANÁLISE DAS MÉTRICAS DE UTILIZAÇÃO DOS CONTAINERS **36**
5.5. COMPARAÇÃO COM A LITERATURA **38**
6. CONCLUSÃO 39
7. REFERÊNCIAS **41**

1. INTRODUÇÃO

Nas últimas décadas, a evolução das tecnologias digitais transformou radicalmente a forma como geramos, coletamos e lidamos com dados, seja ela no contexto cotidiano, nos negócios ou nos ambientes industriais. No setor industrial, esse movimento é intensificado pelo avanço da Indústria 4.0, caracterizada pela integração de sistemas ciber-físicos, sensores distribuídos e conectividade contínua, resultando em uma produção massiva e incessante de dados operacionais (LEE; BAGHERI; KAO, 2015). Mais do que apenas ampliar o volume de informações disponíveis, esse cenário viabiliza aplicações estratégicas como manutenção preditiva, automação de processos e suporte à tomada de decisão.

Entretanto, o aumento da complexidade e da escala dos dados industriais impõe desafios relevantes às arquiteturas responsáveis por sua ingestão e processamento. Ambientes de manufatura inteligente são marcados por dados heterogêneos, gerados em diferentes frequências e escalas temporais, exigindo soluções capazes de lidar simultaneamente com restrições de volume, velocidade e custo computacional (KANG et al., 2016). Nesse contexto, a eficiência da ingestão de dados passa a desempenhar um papel central no desempenho global dos sistemas analíticos.

Dentre as abordagens mais utilizadas para ingestão e processamento de dados, destacam-se duas estratégias amplamente consolidadas: a ingestão em *batch*, que opera sobre blocos de dados acumulados ao longo do tempo, e a ingestão em *stream*, que realiza o processamento contínuo à medida que os dados são gerados. Cada uma dessas abordagens apresenta características distintas em termos de latência, vazão e uso de recursos, o que torna a escolha entre elas uma decisão arquitetural crítica, fortemente dependente do perfil da aplicação e de seus requisitos temporais (BENJELLOUN et al., 2020; KREPS; NARKHEDE; RAO, 2011).

Embora a literatura apresente discussões teóricas consistentes sobre essas estratégias, ainda são limitados os estudos experimentais que realizam comparações sistemáticas entre ingestão em batch e em stream sob condições controladas e realistas, especialmente em cenários industriais baseados em Internet das Coisas (IoT). Avaliações em ambientes reais frequentemente esbarram em

restrições operacionais, o que reforça a importância do uso de ambientes simulados para análise de desempenho e validação arquitetural (MARJANI et al., 2017).

Diante desse contexto, este trabalho propõe uma análise comparativa entre as abordagens de ingestão de dados em batch e em stream por meio de um experimento controlado em um ambiente industrial simulado. O cenário considera uma planta composta por 33 máquinas e 78 sensores, permitindo a avaliação do comportamento das duas estratégias sob diferentes níveis de carga. As análises são conduzidas com base em métricas objetivas, como latência, throughput, uso de CPU, memória e operações de entrada e saída, com o objetivo de fornecer subsídios empíricos para a compreensão dos *trade-offs* envolvidos na escolha do modelo de ingestão mais adequado a aplicações industriais.

2. FUNDAMENTAÇÃO TEÓRICA

2.1. INTERNET DAS COISAS E O DESAFIO DA INGESTÃO DE DADOS

O avanço da Internet das Coisas (*Internet of Things*) impulsionou a geração massiva e contínua de dados provenientes de sensores e dispositivos conectados. Esses dados, geralmente em formato de fluxos não estruturados, são caracterizados por sua alta velocidade, volume e variabilidade. A análise desses fluxos em tempo hábil é essencial para aplicações em cidades inteligentes, monitoramento em tempo real e automação industrial (CAO et al., 2019). Entretanto, trabalhar com dados IoT reais envolve desafios como instabilidade na conexão, custos operacionais e questões éticas.

2.2. INGESTÃO DE DADOS: CONCEITO E CLASSIFICAÇÕES

A ingestão de dados corresponde ao conjunto de processos responsáveis por coletar, transportar e disponibilizar dados provenientes de diferentes fontes para sistemas de armazenamento ou processamento analítico. Em ambientes de IoT e Indústria 4.0, esse processo assume papel central, uma vez que os dados são gerados de forma contínua, distribuída e em diferentes escalas temporais, exigindo mecanismos eficientes para garantir confiabilidade, desempenho e escalabilidade (MARJANI et al., 2017).

Do ponto de vista arquitetural, a ingestão de dados não se limita ao simples recebimento das informações, mas envolve decisões relacionadas ao modelo de processamento, à granularidade temporal, à tolerância a atrasos e ao uso de recursos computacionais. Essas decisões impactam diretamente métricas como latência e *throughput*, influenciando a adequação da solução a diferentes tipos de aplicação industrial (BENJELLOUN et al., 2020).

De forma geral, as estratégias de ingestão podem ser classificadas a partir de diferentes critérios, sendo o mais recorrente na literatura a distinção entre ingestão em lote (*batch*) e ingestão em fluxo contínuo (*stream*). No modelo *batch*, os dados são acumulados ao longo de um intervalo de tempo e processados em conjuntos discretos, priorizando a eficiência no tratamento de grandes volumes e a consistência dos resultados. Em contraste, a ingestão em stream realiza o processamento incremental à medida que os dados são produzidos, privilegiando

baixa latência e resposta quase imediata, ainda que com maior complexidade operacional (CAO et al., 2019).

Dessa forma, compreender os conceitos e classificações da ingestão de dados é fundamental para avaliar o impacto das escolhas arquiteturais sobre o desempenho de sistemas industriais. Essa compreensão fornece a base necessária para a análise comparativa entre os modelos batch e stream, desenvolvida nas seções subsequentes, nas quais os modelos batch e stream são detalhados individualmente.

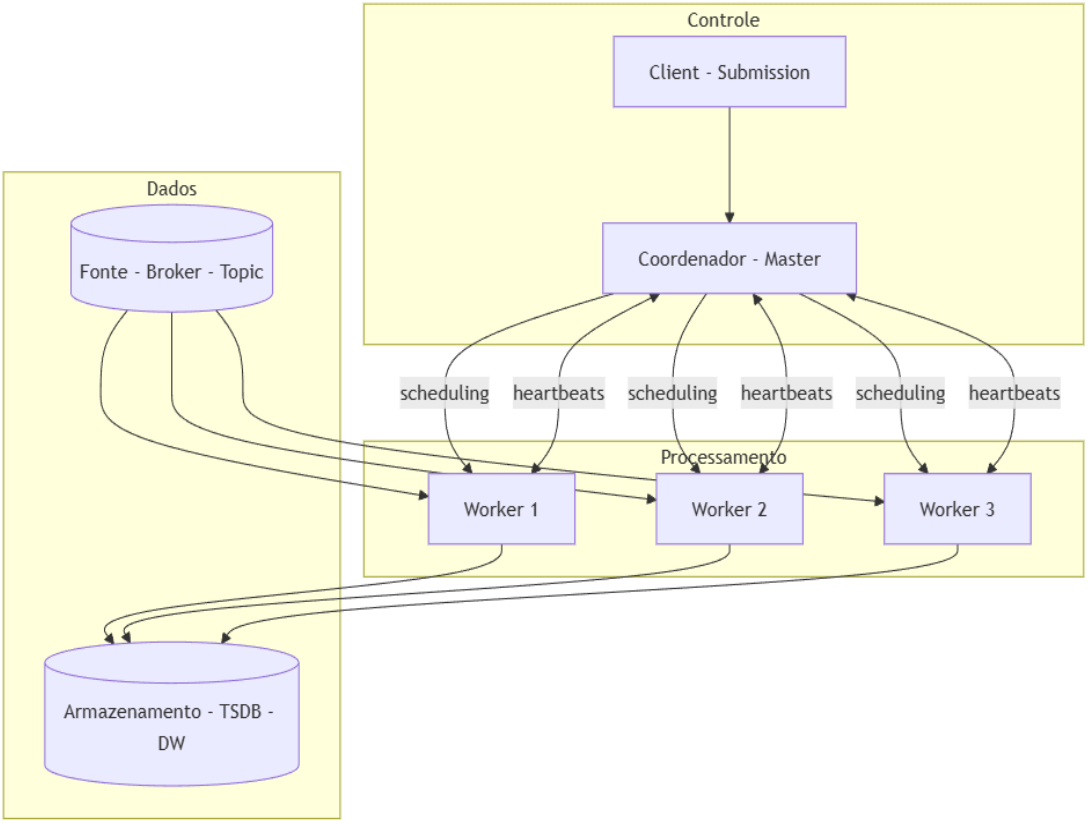


Figura 1: Arquitetura agnóstica de ingestão de dados

A Figura 1 apresenta uma arquitetura de ingestão de dados em alto nível, baseada em um modelo distribuído com separação entre plano de controle e plano de dados. O componente de submissão representa o ponto de inicialização da aplicação, responsável por encaminhar a configuração ao nó coordenador, que por sua vez distribui as tarefas entre múltiplos *workers*. Os dados fluem diretamente das fontes para os *workers*, sendo processados em paralelo e posteriormente persistidos em sistemas de armazenamento analítico.

2.3. PROCESSAMENTO

O processamento em stream é orientado a eventos e permite analisar os dados à medida que chegam ao sistema. Essa abordagem é essencial em cenários onde decisões devem ser tomadas em tempo real, como sistemas de tráfego inteligente ou aplicações financeiras. Seu principal benefício está na baixa latência, porém, há limitações quanto à consistência dos dados e à complexidade na detecção de padrões longos ou comportamentos agregados ao longo do tempo (CAO et al., 2019).

O processamento em lote (batch) executa tarefas sobre grandes volumes de dados acumulados em janelas de tempo definidas. Essa abordagem é tradicionalmente utilizada em contextos analíticos, onde não há urgência na resposta, mas é necessário um alto grau de confiabilidade e profundidade na análise. O uso de frameworks como Apache Hadoop e Spark permite a execução de tarefas paralelas e escaláveis, sendo ideal para modelos de aprendizado offline, como ilustrado no experimento 2 do estudo de Cao et al. (2019), que utilizou dois anos de dados históricos para prever a ocupação de vagas em um sistema de estacionamento inteligente.

2.4. ARQUITETURAS HÍBRIDAS: KAPPA E LAMBDA

A dicotomia entre processamento em lote (batch) e processamento em fluxo (stream) pode ser mitigada por arquiteturas híbridas, que combinam características de ambos os paradigmas para atender simultaneamente demandas de baixa latência e de consistência/completude analítica. Entre os padrões mais citados na literatura, destacam-se as arquiteturas **Lambda** e **Kappa** (TATIPAMULA, 2025).

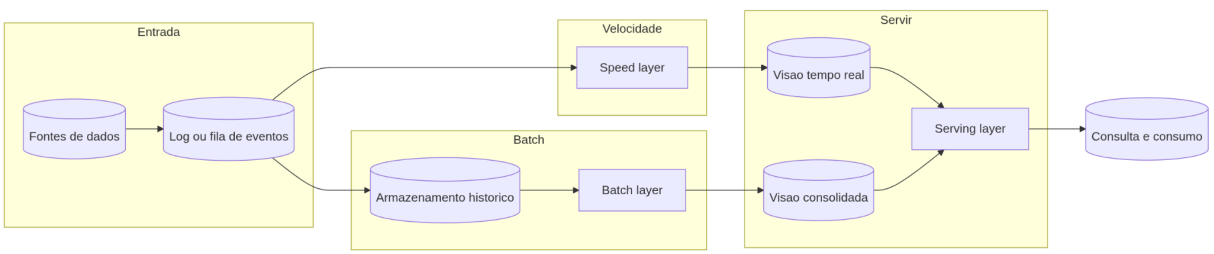


Figura 2: Processamento de dados seguindo a arquitetura Lambda

A arquitetura Lambda (Figura 2) organiza o processamento em múltiplas camadas, combinando uma camada batch (voltada a resultados completos e consistentes) e uma camada de velocidade (speed layer), orientada à geração de visões em tempo quase real. Uma camada de *serving* consolida os resultados, conciliando a imediaticidade do fluxo recente com a consistência do processamento histórico. Apesar de mitigar o *trade-off* entre latência e completude, esse modelo tende a aumentar a complexidade de desenvolvimento e manutenção, em especial pela necessidade de manter lógicas equivalentes em dois caminhos de processamento (TATIPAMULA, 2025).

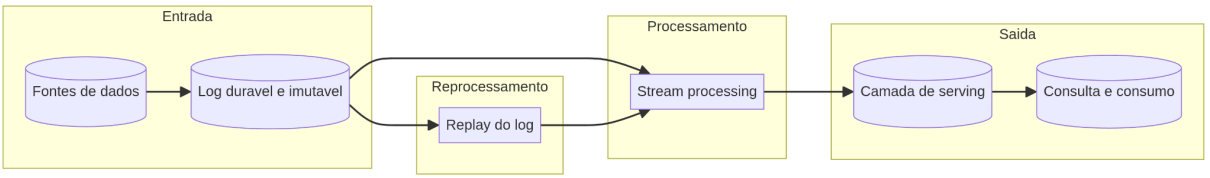


Figura 3: Processamento de dados seguindo a arquitetura Kappa

A arquitetura Kappa, por sua vez, propõe a simplificação do modelo Lambda ao adotar um único caminho de processamento em stream, tratando reprocessamentos históricos como uma forma de “batch equivalente” por meio de replay de um log durável de eventos. Assim, o sistema mantém um registro imutável dos dados de entrada e reexecuta a mesma lógica para dados históricos quando necessário, reduzindo a duplicidade de pipelines e de código (TATIPAMULA, 2025).

No contexto deste trabalho, esses padrões são relevantes por deixarem explícitas decisões arquiteturais que impactam diretamente requisitos de latência, custo operacional e reprocessamento de dados, oferecendo uma base conceitual para discutir cenários em que batch e stream podem coexistir ou ser unificados. Além disso, Benjelloun et al. (2020) destacam a Lambda como alternativa que busca combinar vantagens de volume (batch) e velocidade (stream), reforçando sua pertinência na discussão de abordagens híbridas.

3. TRABALHOS RELACIONADOS

3.1. INTEGRAÇÃO ENTRE PROCESSAMENTO BATCH E STREAM

A integração entre processamento em lote e em fluxo tem sido tema recorrente em soluções voltadas para ambientes IoT. Cao et al. (2019) propõem uma arquitetura baseada em nuvem capaz de operar com dados atuais, obsoletos e históricos em um sistema unificado. No estudo, são utilizados dois fluxos distintos: um em tempo real, com treinamento incremental de modelos de aprendizado de máquina, e outro em lote, com dados históricos para prever a disponibilidade de vagas em um sistema de estacionamento inteligente. A principal contribuição reside na capacidade de combinar análise online e offline em uma mesma infraestrutura, destacando os desafios de latência, redundância de código e inconsistência na semântica dos dados processados.

Benjelloun et al. (2020) realizam uma comparação entre frameworks populares de Big Data, como Hadoop, Spark e Flink. Embora o foco não seja estritamente em IoT, o estudo oferece insights sobre o comportamento de sistemas batch e stream, apontando o Apache Spark como uma solução híbrida promissora devido ao seu suporte a processamento estruturado em tempo real.

3.2. ESTUDOS EM AMBIENTES IOT

Cao et al. (2019) propõem uma arquitetura baseada em nuvem que integra processamento batch e stream para dados IoT, utilizando sensores reais de um sistema de estacionamento inteligente. O estudo avalia o comportamento das duas abordagens sob condições operacionais distintas, observando trade-offs entre latência e completude analítica. Embora o ambiente seja real e não simulado, o trabalho evidencia desafios práticos como redundância de código entre pipelines e inconsistência na semântica dos dados processados, reforçando a necessidade de estudos comparativos em ambientes controlados. O presente trabalho busca endereçar por meio de um cenário simulado com variação sistemática de carga.

O protótipo desenvolvido neste trabalho segue uma linha semelhante, ao implementar dois fluxos independentes de ingestão (stream e batch) sobre um ambiente IoT simulado, possibilitando a observação de métricas sob diferentes condições de carga.

3.3. LIMITAÇÕES OBSERVADAS NOS TRABALHOS EXISTENTES

Apesar das contribuições relevantes, os trabalhos analisados apontam desafios persistentes na adoção de arquiteturas híbridas. Entre eles, destaca-se a dificuldade em consolidar os resultados de pipelines distintas (Cao et al., 2019), a complexidade operacional de manter múltiplos frameworks em paralelo (Tatipamula, 2025).

Essas limitações evidenciam a necessidade de experimentos mais direcionados, especialmente em ambientes controlados, que permitam avaliar o comportamento de diferentes abordagens sob as mesmas condições técnicas e operacionais.

4. METODOLOGIA

A avaliação experimental constitui uma etapa central deste trabalho, pois é por meio dela que se busca analisar, de forma objetiva, o impacto das estratégias de ingestão de dados em *batch* e em *stream* sobre o desempenho de sistemas analíticos em ambientes industriais simulados. Diferentemente de abordagens puramente conceituais, esta avaliação tem como foco a observação empírica do comportamento das arquiteturas sob diferentes condições de carga, permitindo identificar *trade-offs* associados a métricas operacionais relevantes.

Do ponto de vista metodológico, a avaliação foi conduzida em um ambiente controlado e simulado, estratégia amplamente empregada em estudos que investigam arquiteturas de processamento de dados em cenários industriais. Trabalhos como o de Cao et al. (2019) demonstram que ambientes simulados permitem analisar, de maneira sistemática, o comportamento de pipelines batch e stream, reduzindo interferências externas e possibilitando a repetibilidade dos experimentos. De forma semelhante, este estudo adota um desenho experimental

que isola variáveis de interesse, como o volume de dados e o modelo de ingestão, mantendo constantes os demais fatores do ambiente.

A metodologia seguida neste trabalho baseia-se, portanto, em um modelo comparativo, no qual ambas as estratégias de ingestão são avaliadas sob condições equivalentes, variando-se apenas a carga de dados aplicada ao sistema. Esse tipo de abordagem está alinhado a princípios clássicos de avaliação de desempenho de sistemas computacionais, que recomendam o controle das variáveis experimentais e a comparação justa entre diferentes configurações como forma de garantir a validade dos resultados obtidos (JAIN, 1991). Dessa forma, torna-se possível observar como cada modelo responde ao aumento progressivo da demanda, fornecendo subsídios para a análise do impacto das escolhas arquiteturais sobre o desempenho do sistema. Ao estabelecer esse paralelismo metodológico com a literatura existente, a avaliação experimental busca não apenas validar empiricamente os resultados obtidos, mas também contribuir para a compreensão prática dos limites e vantagens das abordagens batch e stream em contextos industriais.

4.1. MÉTRICAS DE AVALIAÇÃO

O principal objetivo da avaliação é comparar as abordagens de ingestão em batch e em stream a partir de métricas mensuráveis, como latência, throughput e consumo de recursos computacionais, considerando cenários com diferentes volumes de dados. A escolha dessas métricas está alinhada com a literatura que discute os desafios da ingestão e do processamento de dados em ambientes de Internet das Coisas (IoT) e Indústria 4.0, nos quais requisitos de desempenho e escalabilidade exercem papel determinante na adequação das soluções adotadas (BENJELLOUN et al., 2020; MARJANI et al., 2017).

A seleção das métricas seguiu o framework proposto por Jain (1991), segundo o qual o desempenho de um sistema pode ser avaliado por três dimensões: o tempo para realizar o serviço, a taxa com que o serviço é realizado, e os recursos consumidos nesse processo, denominadas, respectivamente, métricas de responsividade, produtividade e utilização. No contexto deste trabalho, essas

dimensões correspondem ao delay, ao throughput e ao consumo de CPU e memória.

De acordo com essa abordagem, a análise de desempenho deve considerar de forma integrada a técnica de avaliação adotada, as métricas utilizadas e a carga de trabalho aplicada ao sistema. As métricas representam os critérios quantitativos empregados para avaliar o comportamento do sistema, enquanto a carga de trabalho corresponde ao conjunto de requisições ou eventos aos quais o sistema é submetido. Nesse contexto, métricas como latência e throughput são amplamente empregadas para caracterizar, respectivamente, o tempo de resposta e a capacidade de processamento de sistemas sob diferentes níveis de demanda.

A tabela 1 apresenta as métricas de responsividade e produtividade. O *throughput* representa a quantidade de linhas que são recebidas e processadas. Já o *delay,* representa o tempo necessário entre a criação do dado na fonte (sensores) até o processamento e escrita no banco.

Métrica	**Descrição**
Delay	Intervalo entre o timestamp de criação do dado na fonte e o timestamp de confirmação de escrita no banco
Throughput	Volume de registros processados e persistidos por unidade de tempo

Tabela 1: Métricas de responsividade e produtividade

A Tabela 2 apresenta as métricas de utilização, coletadas a partir dos logs de monitoramento dos containers Docker.

Métrica	**Descrição**
Uso de CPU	Percentual médio de utilização do processador por container
Uso de Memória	Consumo médio de memória RAM por container

Tabela 2: Métricas de utilização

4.2. FERRAMENTAS E PARÂMETROS UTILIZADOS

Os parâmetros do experimento são divididos em dois grupos: aqueles mantidos constantes ao longo de todas as execuções, que caracterizam o ambiente e a infraestrutura, e aqueles deliberadamente variados, que constituem os fatores experimentais propriamente ditos.

Todos os experimentos foram executados na mesma máquina host, com a mesma configuração de rede interna entre containers. Cada execução teve duração de 60 minutos. O ambiente simulado compreendeu 78 sensores distribuídos em 33 máquinas, com frequências de envio individuais variando entre 0,5 Hz e 5 Hz, resultando em uma taxa base de aproximadamente 105 registros por segundo. O intervalo de execução do processo batch foi fixado em 5 minutos.

A Tabela 3 apresenta as ferramentas utilizadas, seus papéis no experimento e a justificativa objetiva de cada escolha.

Componente	**Ferramenta**	**Justificativa**
Linguagem de desenvolvimento	Python	Suporte nativo a paradigmas assíncronos, bibliotecas maduras para MQTT, bancos de dados e manipulação de dados
Protocolo de mensageria	MQTT	Protocolo leve, padrão de facto em IoT, adequado para fluxos contínuos com restrição de overhead
Banco de dados (Stream)	InfluxDB	Banco orientado a séries temporais, otimizado para escrita de alta frequência e consultas temporais
Banco de dados (Batch)	PostgreSQL	Banco relacional consolidado, adequado para inserções em bloco e análises retrospectivas via SQL
Visualização e monitoramento	Grafana	Integração nativa com InfluxDB e PostgreSQL, permitindo comparação unificada das métricas sob o mesmo painel
Orquestração	Docker / Docker Compose	Isolamento de ambiente, reprodutibilidade e controle de recursos por container

Tabela 3: Ferramentas e justificativas

O único fator variado entre as execuções foi o multiplicador de carga (*RATE_MODIFIER*), aplicado uniformemente sobre a frequência de todos os sensores. Foram avaliados três níveis: 1x (carga base), 2x (carga dobrada) e 4x (carga quadruplicada). Esses níveis foram escolhidos para permitir a observação do comportamento de cada abordagem tanto em condições normais de operação quanto sob estresse progressivo, sem que o volume extrapolasse a capacidade física da infraestrutura de teste. A abordagem de ingestão (batch ou stream) constitui o segundo fator, avaliado em paralelo para cada nível de carga.

4.3. DESIGN DO EXPERIMENTO

O experimento foi projetado com o objetivo de comparar o desempenho de dois métodos de ingestão de dados em ambientes de Internet das Coisas (*IoT*): ingestão em tempo real (*stream*) e ingestão em pacotes (*batch*). A arquitetura proposta simula uma planta industrial, composta por dezenas de sensores distribuídos entre diferentes máquinas, enviando dados com diferentes frequências e formatos.

Para garantir reprodutibilidade e isolamento, todos os componentes foram conteinerizados utilizando *Docker*, incluindo banco de dados, geradores, consumidores e ferramentas de monitoramento. O controle e orquestração dos serviços foi feito por meio do *docker-compose*.

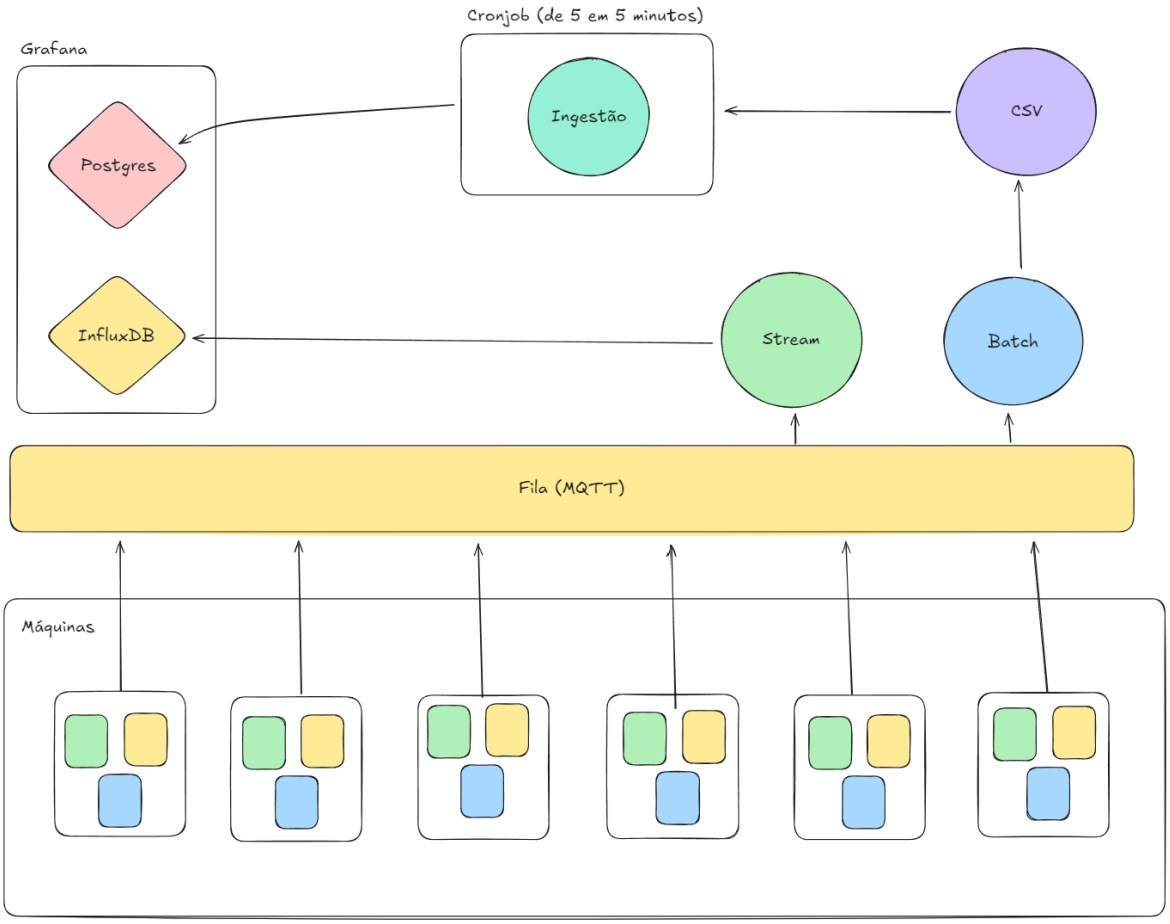


Figura 4: Desenho descritivo da arquitetura do experimento

As etapas do experimento foram definidas da seguinte forma:

4.3.1. Simulação da planta industrial

A geração de dados foi implementada em Python, utilizando asyncio e threading para simular o comportamento concorrente de múltiplos sensores industriais. Cada sensor publica leituras periódicas (pressão, temperatura, vibração, nível e status) em um único tópico MQTT, direcionado a um broker Eclipse Mosquitto local. O payload de cada mensagem inclui os campos *created_at* e *ingested_at*, que viabilizam o cálculo do delay por registro na etapa de análise.

4.3.2. Ingestão dos dados

Dois consumidores independentes operam sobre o mesmo tópico MQTT:

- Stream: Os dados são consumidos em tempo real por um consumidor Python que escreve diretamente no banco de dados InfluxDB. O tempo

de recebimento e o tempo de ingestão são armazenados como campos separados.

- Batch: Os dados recebidos são acumulados em arquivos CSV e processados em lotes por um consumidor, que os insere no banco de dados. Esse consumidor é um processo separado, agendado para ser executado a cada 5 minutos.

A separação entre os dois pipelines no Docker garante que o comportamento de um não interfira nas métricas do outro.

4.3.3. Coleta de métricas e instrumentação

O desempenho de cada abordagem é avaliado mediante o cálculo das seguintes métricas: *Delay* por registro que mensura o intervalo entre o momento de criação da mensagem e sua efetiva ingestão. *Throughput que* quantifica o volume de registros processados por minuto. E, por fim, o desvio padrão do *delay* que é utilizado para verificar o nível de estabilidade durante o processo de ingestão. O Grafana foi utilizado para monitoramento em tempo de execução, por meio de painéis conectados diretamente às duas fontes de dados, permitindo a identificação de anomalias durante os testes.

4.3.4. Execução com diferentes cargas de trabalho

A frequência de envio dos sensores pode ser modificada dinamicamente, simulando diferentes cenários de carga. Essa variação permite avaliar como cada método responde ao aumento no volume de dados.

Este design permite observar, de forma controlada, como as estratégias de ingestão se comportam em termos de latência, estabilidade, escalabilidade e consumo de recursos, fornecendo insumos sólidos para a análise comparativa dos resultados.

5. DISCUSSÃO DOS RESULTADOS

5.1. COMPARATIVO ENTRE ABORDAGENS COM CARGA DE 1x

Os testes com carga de 1x visam observar o comportamento inicial das abordagens batch e stream diante de um volume reduzido de dados. A análise contempla as métricas de CPU, memória, throughput e delay, fornecendo subsídios para

compreender os impactos do modelo de processamento adotado.

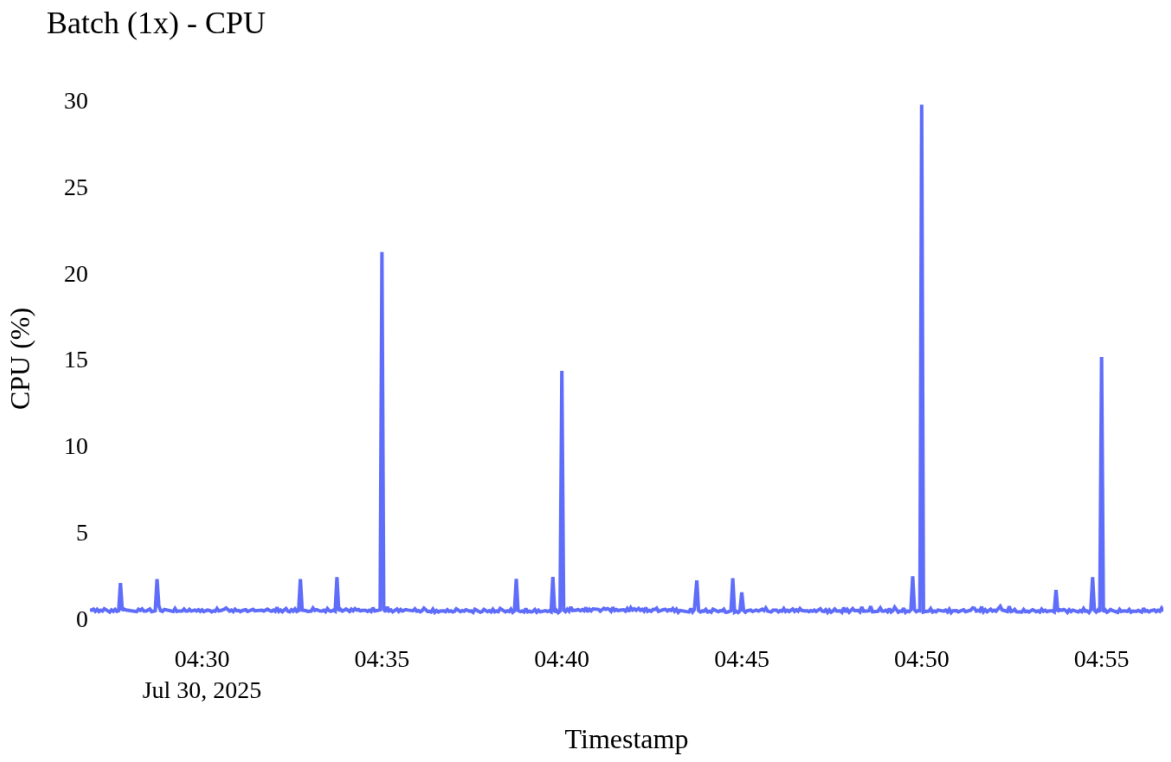


Figura 5: Gráfico comparativo de uso de CPU pelo tempo. Experimento Batch (1x)

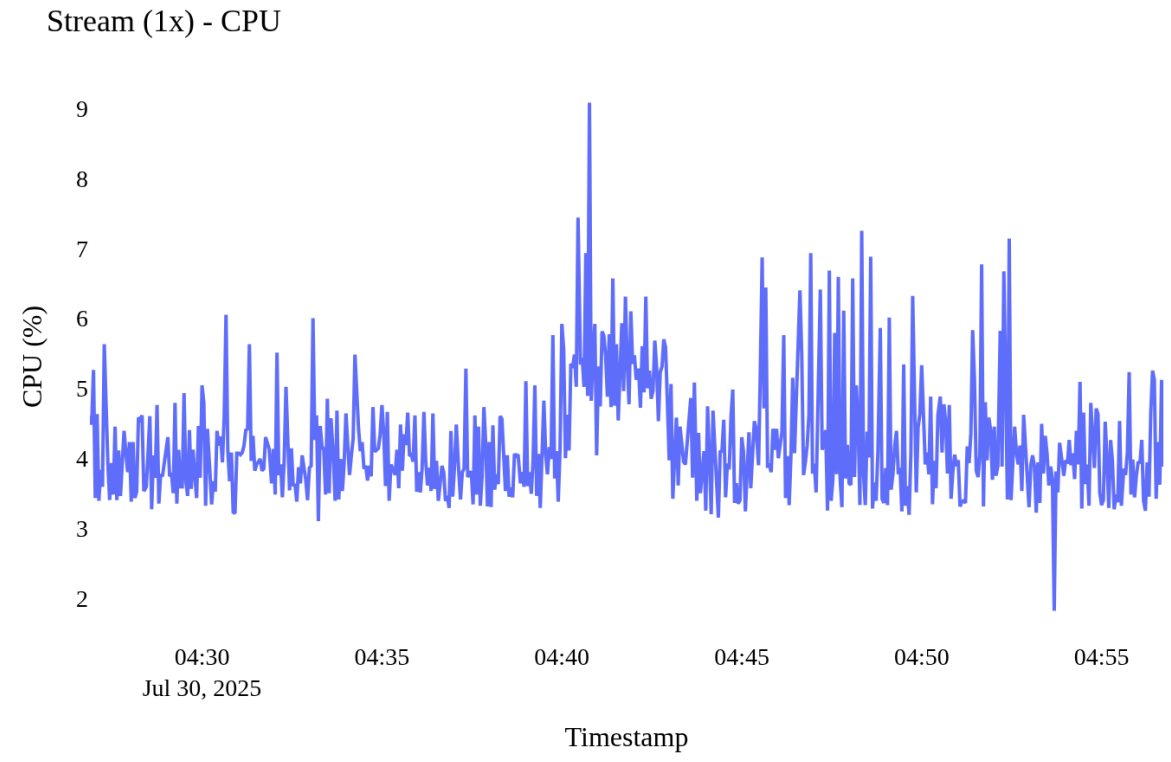


Figura 6: Gráfico comparativo de uso de CPU pelo tempo. Experimento Batch (1x)

No uso de CPU, a abordagem batch apresentou picos esporádicos, atingindo até 30% em momentos de estresse extremo, intercalados por longos períodos de baixa atividade (figura 5). Esse comportamento reflete a natureza do modelo, que processa os dados em blocos. Em contraste, o modelo stream demonstrou um consumo mais linear, variando entre 4% e 9%, com uso mais constante ao longo do tempo.

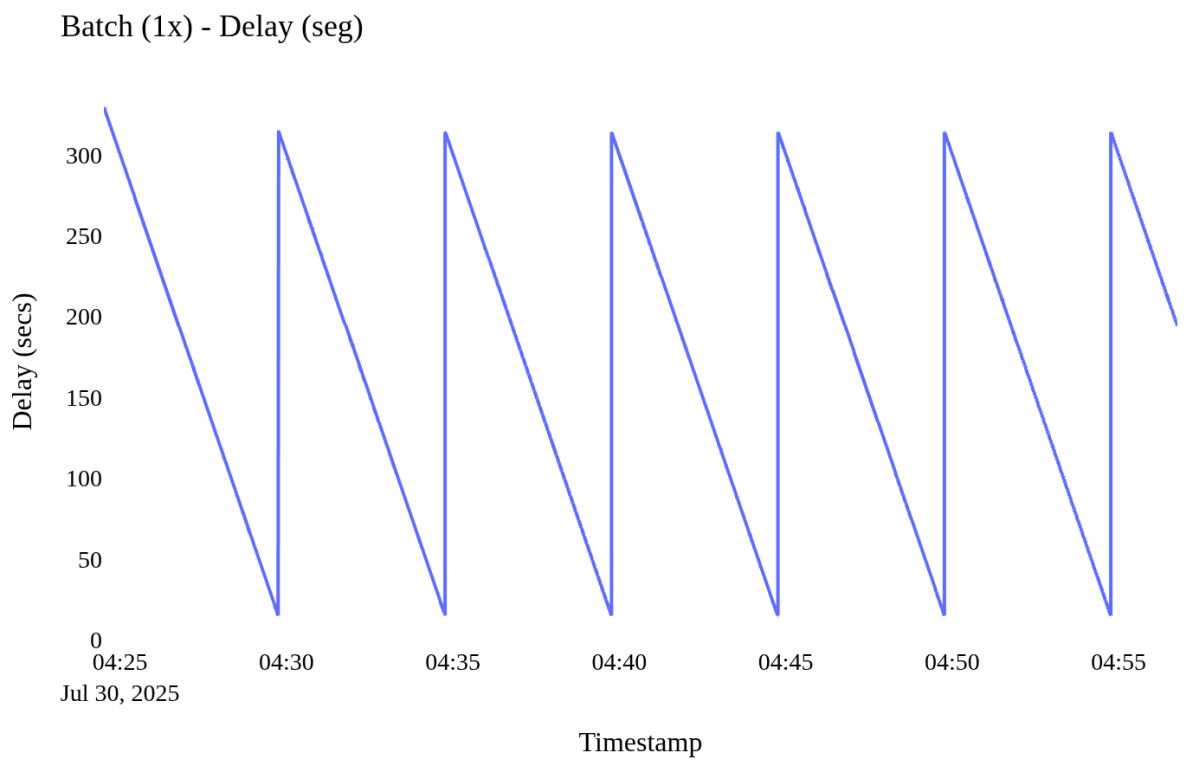


Figura 7: Representação da visão de Delay pelo tempo (Batch - 1x)

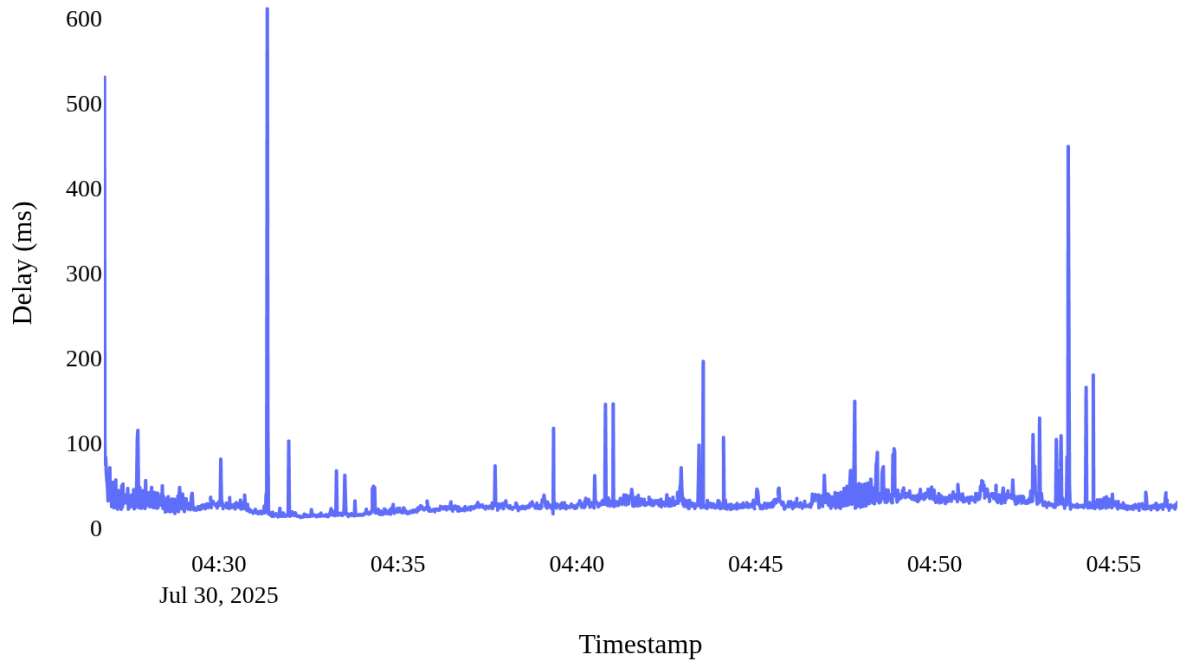


Figura 8: Representação da visão de Delay pelo tempo (Stream - 1x)

Quanto ao delay, visualizado nas figuras 7 e 8, a diferença entre as abordagens é notável. O batch apresentou picos superiores a 300 segundos, indicando a espera acumulada entre as execuções dos lotes. Já o stream manteve o delay abaixo de 200 milissegundos (exceto picos esporádicos), evidenciando maior agilidade na entrega dos dados processados. Esse comportamento é coerente com o modelo teórico de Das et al. (2014), segundo o qual intervalos de batch reduzidos tendem a minimizar a latência enquanto a carga permanece dentro da capacidade do sistema.

Em termos de uso de memória, ambas as abordagens apresentaram baixo consumo. O batch oscilou levemente em torno de 0,09%, enquanto o stream manteve uso praticamente estável durante todo o período, o que reforça sua eficiência nesse aspecto.

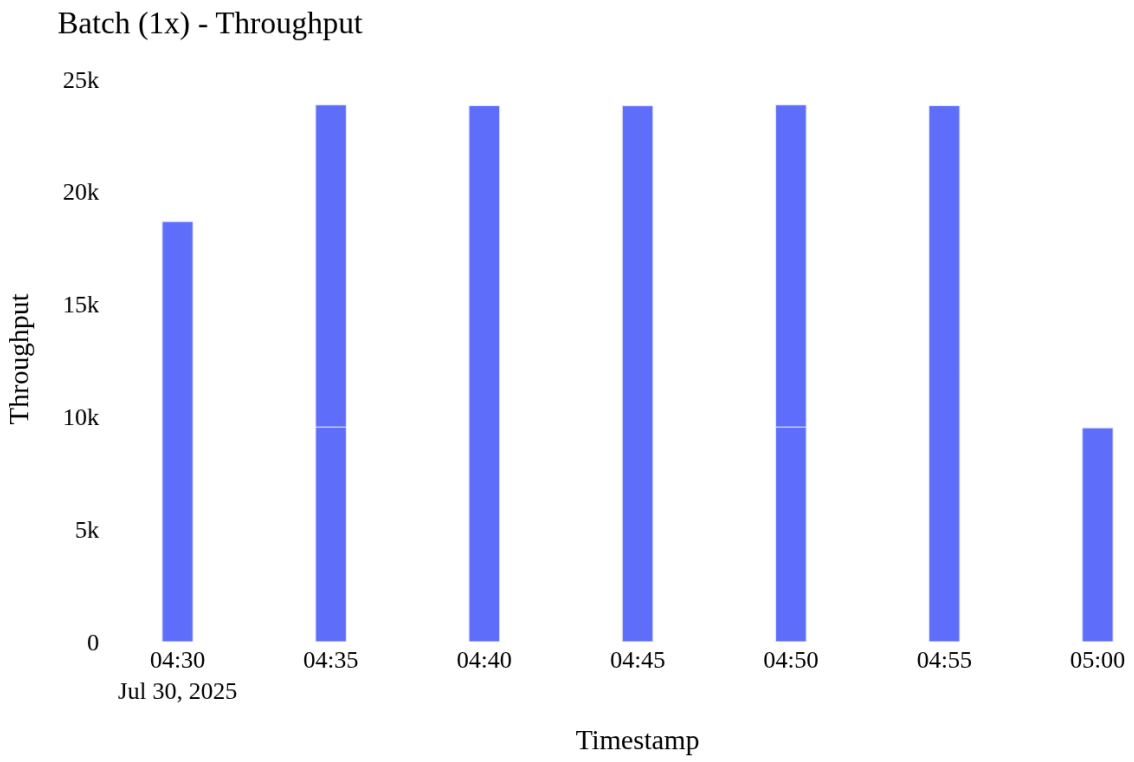


Figura 9: Representação gráfica do Throughput pelo tempo (Batch - 1x)

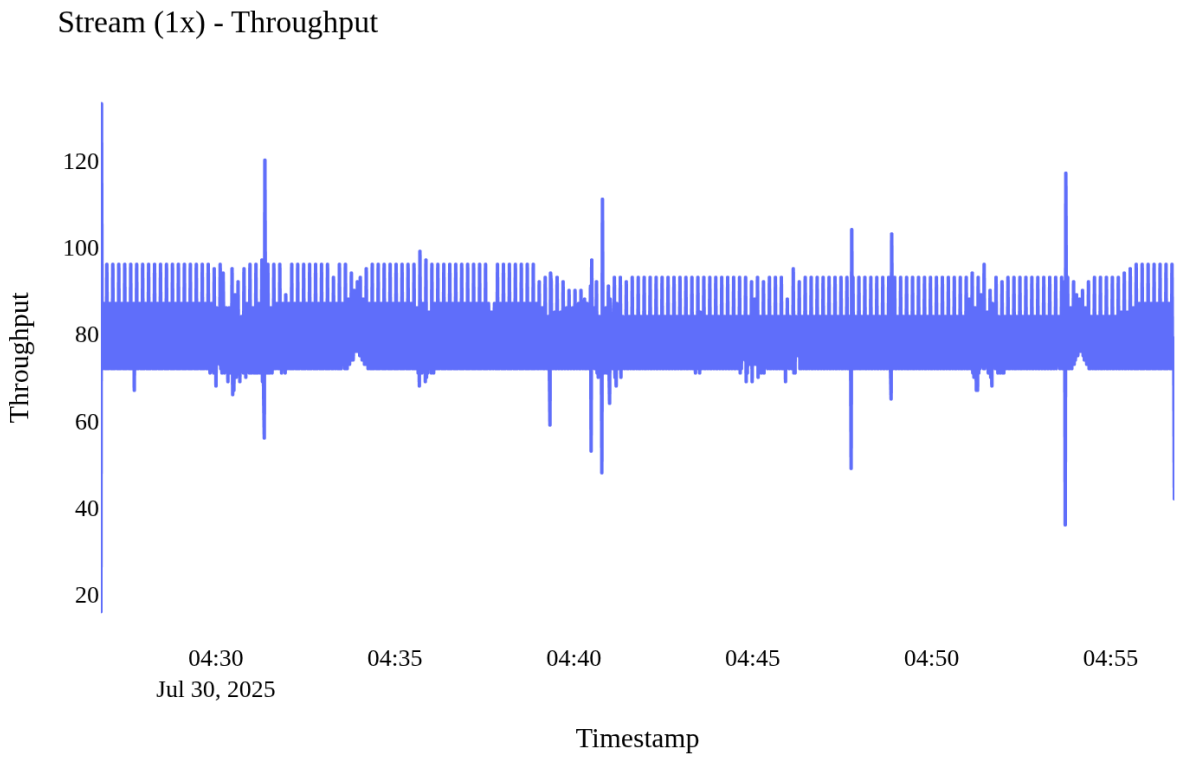


Figura 10: Representação gráfica do Throughput pelo tempo (Stream - 1x)

A métrica de throughput revelou abordagens distintas. Como podemos observar nas imagens 9 e 10, a abordagem em batch apresentou picos superiores a 20 mil eventos por janela de tempo, mas com interrupções entre os ciclos. Já o stream manteve uma taxa constante entre 70 e 100 eventos por segundo, favorecendo aplicações que exigem respostas contínuas.

Dessa forma, a análise da carga de 1x evidencia que, embora o batch seja eficiente em termos de volume processado por ciclo, o stream se destaca na entrega em tempo real, com maior regularidade e menor latência.

5.2. COMPARATIVO ENTRE ABORDAGENS - CARGA DE 2X

Com o dobro do volume de dados da carga anterior, é possível observar mudanças mais acentuadas no comportamento das abordagens *batch* e *stream*. A elevação da carga acentua diferenças e ressalta características inerentes a cada modelo de processamento.

Na utilização de CPU, o *batch* manteve seu padrão de picos isolados, atingindo valores superiores a 25%. Estes picos ocorrem durante a execução dos lotes, seguidos por períodos de baixa atividade. Já o *stream* registrou uso contínuo da CPU, com variações entre 12% e 22%, mantendo o padrão observado na carga de 1x, mas com picos mais pronunciados.

A latência do *batch* aumentou significativamente. O *delay* se manteve próximo de 300 segundos antes de cada execução, refletindo o tempo de espera entre os ciclos. Por outro lado, o *stream* continuou operando com latência reduzida, oscilando entre 100 e 800 milissegundos. Apesar do leve aumento, o desempenho permanece dentro do esperado.

Em relação à memória, o *batch* apresentou crescimento progressivo, com saltos discretos ao longo do tempo, atingindo 0,12% de uso. O *stream* mais uma vez se destacou pela estabilidade, mantendo uso constante e ligeiramente superior ao da carga de 1x.

No *throughput*, o *batch* processou grandes volumes por lote, superando os 80 mil eventos em janelas específicas. Contudo, manteve os intervalos de ociosidade entre execuções. O *stream* atingiu picos de até 370 eventos por segundo, com variações perceptíveis, mas mantendo o fluxo contínuo.

A análise sob carga de 2x reforça o comportamento esperado: o modelo *batch* mostra maior volume por ciclo, porém com latência elevada, enquanto o *stream* mantém desempenho mais estável, mesmo com aumento da carga.

5.3. COMPARATIVO ENTRE ABORDAGENS - CARGA DE 4X

Com a carga quadruplicada em relação ao cenário base, o objetivo foi avaliar a escalabilidade das abordagens *batch* e *stream* e identificar possíveis gargalos sob estresse elevado. A análise reforça as tendências observadas anteriormente, agora com efeitos mais intensos.

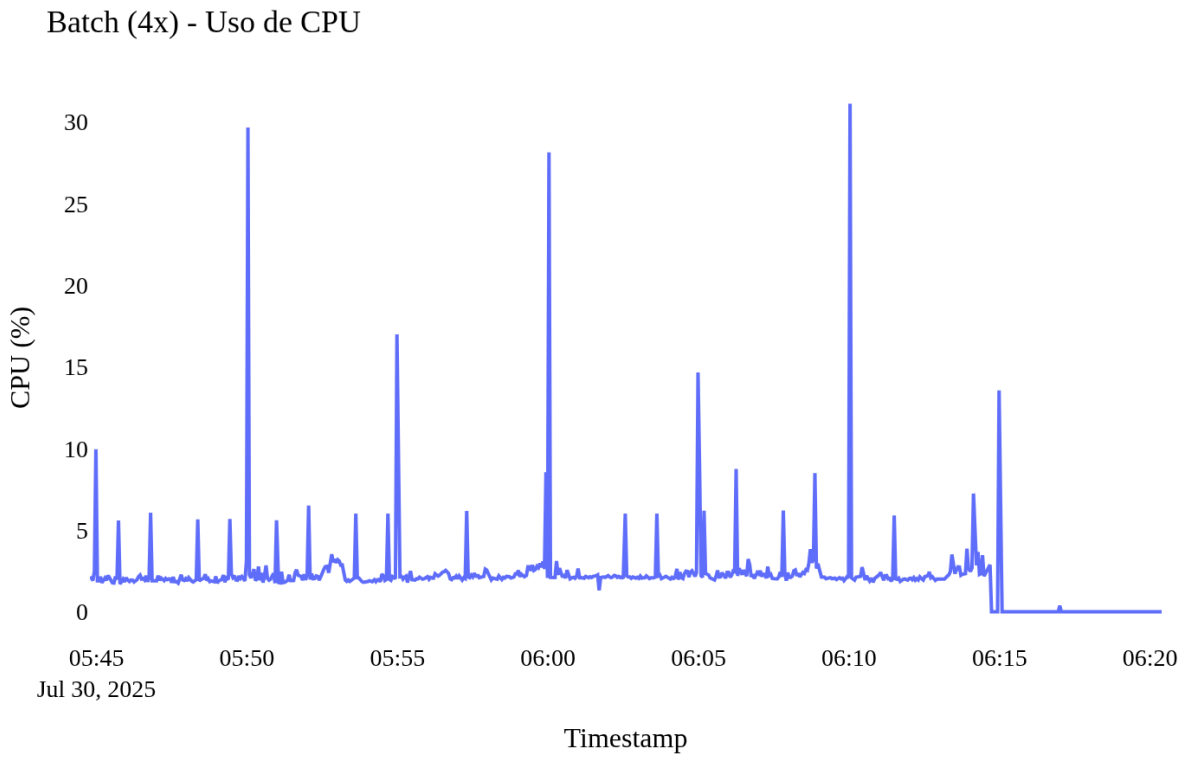


Figura 10: Gráfico representando a relação de uso de CPU pelo Tempo (Batch - 4x)

O uso de CPU (figura 10) na abordagem *batch* apresentou picos frequentes, com valores chegando a 30%, seguidos por períodos de quase inatividade. Esse comportamento evidencia a concentração do processamento em momentos pontuais. Em contrapartida, o *stream* operou com uso constante da CPU, entre 15% e 25%, com oscilações intensas, mas sem inatividade, o que demonstra maior regularidade no consumo de recursos.

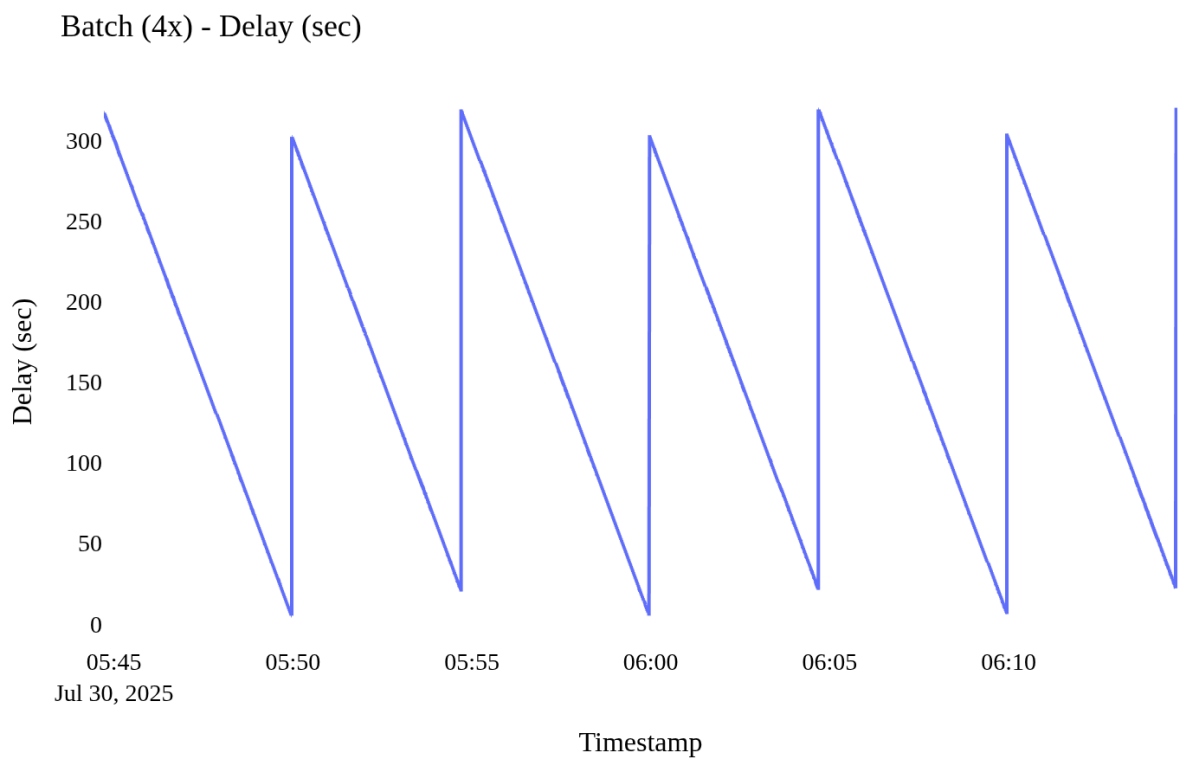


Figura 11: Gráfico representando a relação de Delay pelo Tempo (Batch - 4x volume)

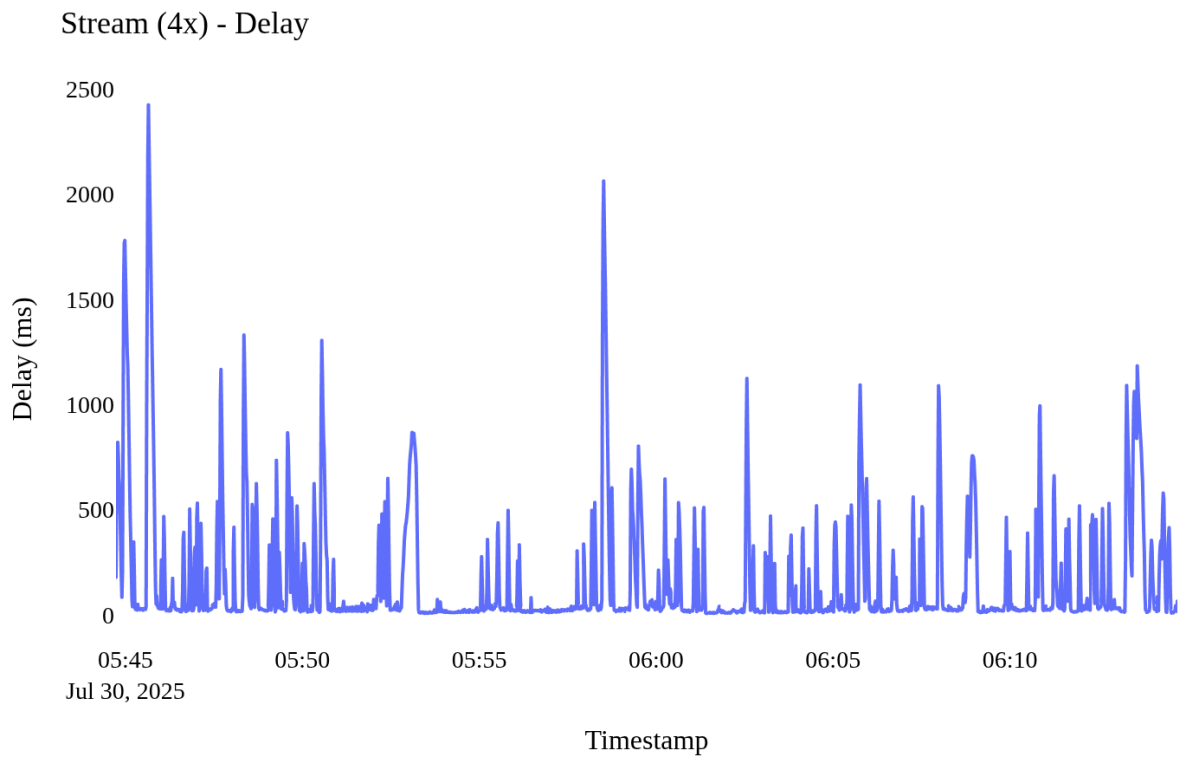


Figura 12: Gráfico representando a relação de Delay pelo Tempo (Stream - 4x)

O *delay* na abordagem *batch* (figura 11) manteve o mesmo padrão cíclico, alcançando valores próximos de 320 segundos antes de cada execução. Isso indica que, mesmo com o aumento da carga, o tempo entre os lotes permaneceu alto. Já o stream (figura 12) apresentou latência mais variável, com picos que ultrapassaram 2.400 milissegundos, embora a maior parte das amostras tenha se mantido abaixo de 500 milissegundos. Esse comportamento é consistente com o observado por Das et al. (2014), que demonstram que à medida que a taxa de dados se aproxima da capacidade de processamento disponível, a latência cresce de forma não-linear, com degradações abruptas nos percentis mais altos.

No consumo de memória, a abordagem *batch* mostrou crescimento gradual ao longo da execução, atingindo 0,18%. Essa elevação, ainda que modesta, reflete o acúmulo de dados entre os ciclos. O *stream* manteve sua característica de estabilidade, sem variações perceptíveis, mesmo diante do volume aumentado.

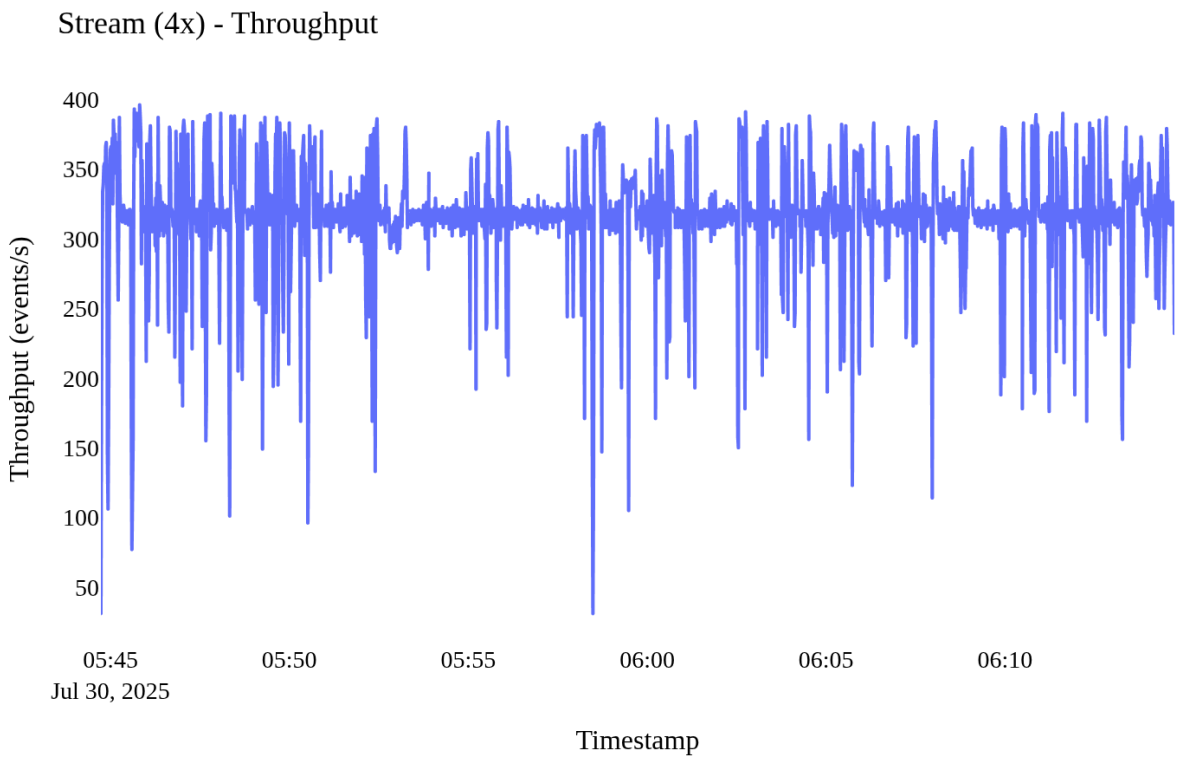


Figura 13: Gráfico representando a relação de Throughput pelo Tempo (Stream - 4x)

Quanto ao *throughput*, o *batch* registrou os maiores volumes até então, processando até 100 mil eventos por lote. Apesar do alto desempenho pontual, os períodos entre lotes continuam a representar inatividade. O *stream (Figura 13)* manteve fluxo constante, com picos superiores a 390 eventos por segundo, ainda que com oscilações mais visíveis do que nas cargas anteriores.

Portanto, sob carga de 4x, as limitações do *batch* em cenários que exigem responsividade tornam-se mais evidentes, enquanto o *stream* comprova sua capacidade de manter o processamento ativo e responsivo, mesmo sob volumes mais elevados.

5.4. ANÁLISE DAS MÉTRICAS DE UTILIZAÇÃO DOS CONTAINERS

A análise dos dados obtidos a partir dos logs do Docker permite avaliar o comportamento dos containers durante a execução dos experimentos com cargas de 1x, 2x e 4x. Foram consideradas as métricas médias de uso de CPU (%) e memória (MiB) para cada um dos principais componentes do ambiente.

O container referente à aplicação de consumo em lote (*batch_consumer*) apresentou crescimento progressivo no uso de CPU e memória. Com carga de 1x, o uso médio de CPU foi de 0,56%, subindo para 1,24% em 2x e alcançando 2,16% em 4x. O uso de memória seguiu a mesma tendência, partindo de 24 MiB e chegando a 36,8 MiB. Esse comportamento reflete a necessidade de recursos proporcionais ao volume de dados processados em cada lote.

No caso da aplicação de consumo contínuo, os dados estão agregados ao container do banco de dados *InfluxDB*, responsável pelo armazenamento das medições. Esse container apresentou a maior sensibilidade ao aumento da carga. O uso médio de CPU subiu de 19,9% na carga de 1x para 38,4% na carga de 4x. O consumo de memória mais do que triplicou, passando de 192 MiB para 611 MiB, evidenciando a intensificação do fluxo de dados gravados continuamente.

O serviço de mensageria *Mosquitto* apresentou baixo impacto em todas as cargas, com uso de CPU variando de 0,29% a 1,12%, e memória em torno de 1,4 MiB. Esses valores indicam que o volume de mensagens, mesmo em 4x, não saturou o serviço. A estabilidade reforça a adequação do MQTT como protocolo leve para cenários IoT.

Os containers *PostgreSQL* e *Grafana* também demonstraram escalabilidade controlada. O *PostgreSQL*, usado na abordagem *batch*, teve aumento no uso de CPU de 1,59% (1x) para 2,84% (4x), e uso de memória de 41,5 MiB para 85,2 MiB. Já o *Grafana*, presente em ambas as abordagens, apresentou crescimento moderado, com CPU oscilando entre 0,85% e 1,54% e memória entre 134 e 185 MiB.

De modo geral, a análise das métricas dos containers confirma a expectativa de aumento proporcional no consumo de recursos com a elevação da carga. A abordagem *stream*, embora mais constante, demandou mais do subsistema de armazenamento, enquanto a abordagem *batch* distribuiu o consumo entre o processador de lotes e o banco relacional.

5.5. COMPARAÇÃO COM A LITERATURA

Os resultados obtidos nos experimentos estão alinhados com os achados descritos na literatura, especialmente no que se refere ao comportamento distinto entre os modelos de processamento em lote e em fluxo contínuo.

Benjelloun et al. (2020) destacam que abordagens *streaming* são mais eficazes em ambientes que exigem baixa latência e resposta imediata. Tal observação é coerente com os dados obtidos neste trabalho, nos quais o *stream* manteve *delay* inferior a 1 segundo, mesmo sob cargas elevadas. Em contraste, o *batch* apresentou latências sistematicamente superiores a 300 segundos, o que compromete sua aplicação em cenários sensíveis ao tempo. Além disso, o uso de containers permitiu observar que o modelo *streaming* tende a sobrecarregar o subsistema de armazenamento, enquanto o *batch* apresenta picos de uso pontuais nos processadores e bancos relacionais.

Outro ponto é a dificuldade de conciliar alto throughput com latência reduzida em sistemas baseados em *batch,* o que comunica bem com o que é defendido por Benjelloun et al. (2020), que distingue os processamentos como *batch* sendo orientado a volume, enquanto *stream* é orientado a velocidade. Isso também foi evidenciado nos experimentos, nos quais a abordagem *batch* obteve alta vazão apenas em janelas isoladas, mas com custo elevado de tempo de espera entre ciclos.

Tatipamula (2025) aponta que sistemas de streaming demandam provisionamento para picos de carga. Complementarmente, Das et al. (2014) demonstram empiricamente que a latência cresce de forma não-linear à medida que o volume se aproxima da capacidade disponível, com degradações especialmente pronunciadas nos momentos em que o tempo de processamento dos lotes supera o intervalo configurado. Esse comportamento foi observado nos experimentos sob carga de 4x, nos quais o stream manteve operação contínua, mas com picos de latência significativamente acima da média.

De forma geral, os dados obtidos confirmam as limitações e vantagens discutidas nos estudos analisados. A abordagem *streaming* oferece maior estabilidade e tempo de resposta, enquanto o *batch* é mais eficiente em

processamentos agregados e análises históricas. Essa distinção é fundamental para orientar a escolha do modelo conforme o perfil da aplicação.

5.6. LIMITAÇÕES

Este estudo foi conduzido em um ambiente simulado, com o objetivo de reproduzir, de forma prática e controlada, o comportamento de uma planta industrial. Embora o modelo de sensores tenha sido desenhado para refletir cenários realistas (tanto em frequência quanto em variabilidade dos dados), a ausência de ruídos externos e instabilidades presentes em ambientes reais constitui uma limitação importante no que diz respeito à generalização dos resultados.

As comparações entre as estratégias de ingestão foram realizadas com uma única configuração de hardware e rede, o que significa que os resultados obtidos podem não se manter sob infraestruturas diferentes, como máquinas com múltiplos núcleos ou arquiteturas distribuídas. Da mesma forma, somente duas tecnologias específicas foram avaliadas: InfluxDB, para ingestão em *stream*, e PostgreSQL, para ingestão em *batch*. Isso implica que as conclusões não podem ser automaticamente extrapoladas para outras ferramentas que adotem os mesmos paradigmas de ingestão.

Ainda que todas as métricas de uso de CPU, memória, *throughput* e latência tenham sido monitoradas continuamente, não foram realizados testes de perda de pacotes, consistência de dados ou tempo de resposta ao consumidor final, uma vez que tais aspectos não faziam parte do escopo do tipo de teste aplicado.

O experimento foi executado por 60 minutos em cada cenário, com testes sob três níveis de carga de trabalho: normal (1x), dobrado (2x) e quadruplicado (4x). O *rate limit* dos sensores foi fixado nesses níveis, o que restringe a avaliação a uma faixa específica de estresse do sistema.

6. CONCLUSÃO

Este trabalho avaliou o comportamento de duas abordagens analíticas, *batch* e *streaming*, em um ambiente simulado de Internet das Coisas (IoT). Foram analisadas métricas de desempenho, como uso de CPU, memória, *throughput* e *delay*, sob diferentes cargas de dados e condições operacionais.

Os resultados demonstraram que a abordagem *streaming* apresentou maior estabilidade, menor latência e maior adequação para aplicações com requisitos de tempo real. Mesmo com aumento de carga, o *stream* manteve operação contínua, ainda que com oscilações pontuais em cenários mais exigentes.

Por outro lado, a abordagem *batch* obteve desempenho superior em volume processado por janela, sendo mais eficiente para cargas acumuladas e análises periódicas. Contudo, seu *delay* elevado e uso pontual de recursos a tornam menos apropriada para sistemas sensíveis ao tempo.

As métricas extraídas dos containers confirmaram os padrões esperados. O modelo *streaming* demandou mais do subsistema de armazenamento, enquanto o *batch* gerou picos localizados de uso de CPU e memória. A utilização de containers contribuiu para o isolamento e controle das aplicações durante os testes.

Comparando-se com a literatura, os resultados reforçam as vantagens do modelo *streaming* em aplicações que exigem responsividade e escalabilidade. A abordagem *batch*, embora eficaz em processamento massivo, apresenta limitações em cenários com exigência de tempo de resposta.

Como trabalhos futuros, sugere-se a incorporação de mecanismos de balanceamento de carga, avaliação de arquiteturas híbridas e experimentos em ambientes distribuídos. Tais evoluções podem ampliar o escopo da análise e validar os resultados em contextos mais próximos de aplicações reais.

7. REFERÊNCIAS

S. Benjelloun et al., **"Big Data Processing: Batch-based processing and stream-based processing,"** 2020 Fourth International Conference On Intelligent Computing in Data Sciences (ICDS), Fez, Morocco, 2020, pp. 1-6, doi: 10.1109/ICDS50568.2020.9268684.

H. Cao, M. Brown, L. Chen, R. Smith and M. Wachowicz, **"Lessons Learned from Integrating Batch and Stream Processing using IoT Data,"** 2019 Sixth International Conference on Internet of Things: Systems, Management and Security (IOTSMS), Granada, Spain, 2019, pp. 32-34, doi: 10.1109/IOTSMS48152.2019.8939232.

CLOUD NATIVE COMPUTING FOUNDATION. *Cloud Native Interactive Landscape*. Disponível em: https://landscape.cncf.io/. Acesso em: 10 de jan. de 2026

JAIN, Raj. **The art of computer systems performance analysis: techniques for experimental design, measurement, simulation, and modeling**. New York: John Wiley & Sons, 1991.

KANG, H. S. *et al.* **Smart manufacturing: Past research, present findings, and future directions**. *International Journal of Precision Engineering and Manufacturing – Green Technology*, v. 3, n. 1, p. 111–128, 2016. DOI: https://doi.org/10.1007/s40684-016-0015-5

KREPS, J.; NARKHEDE, N.; RAO, J. **Kafka: A distributed messaging system for log processing**. In: *Proceedings of the NetDB*. Athens, Greece, 2011.
Disponível em: https://notes.stephenholiday.com/Kafka.pdf. Acesso em: 10 de jan. de 2026.

LEE, J.; BAGHERI, B.; KAO, H. A. **A cyber-physical systems architecture for Industry 4.0-based manufacturing systems**. *Manufacturing Letters*, v. 3, p. 18–23, 2015. DOI: https://doi.org/10.1016/j.mfglet.2014.12.001

MARJANI, M. *et al.* **Big IoT data analytics: architecture, opportunities, and open research challenges**. *IEEE Access*, v. 5, p. 5247–5261, 2017. DOI: https://doi.org/10.1109/ACCESS.2017.2689040

TATIPAMULA, S. **Real-time vs. batch data processing: when speed matters**. *World Journal of Advanced Research and Reviews*, v. 26, n. 1, p. 1612–1631, 2025.

MQTT. Disponível em: https://mqtt.org. Acesso em: 22 de maio de 2026.

Eclipse Mosquitto. https://mosquitto.org. Acesso em: 22 de maio de 2026.

InfluxDB. https://www.influxdata.com. Acesso em: 22 de maio de 2026.

GRAFANA. https://grafana.com. Acesso em: 22 de maio de 2026.

DOCKER. https://www.docker.com. Acesso em: 22 de maio de 2026.

DAS, Tathagata et al. **Adaptive stream processing using dynamic batch sizing.** In: ACM SYMPOSIUM ON CLOUD COMPUTING (SoCC), 5., 2014, Seattle. Anais [...]. New York: ACM, 2014. DOI: https://doi.org/10.1145/2670979.2670995.